

Docker para Laboratório de Banco de Dados (LBD)

Guia Prático para Alunos de Laboratório de Banco de Dados (LBD)

Sumário

1. [O que é Docker?](#)
 2. [Conceitos Fundamentais](#)
 3. [Instalando o Docker](#)
 4. [Comandos Essenciais do Docker](#)
 5. [MySQL com Docker](#)
 6. [phpMyAdmin com Docker](#)
 7. [Conectando o phpMyAdmin ao MySQL](#)
 8. [Usando Docker Compose \(Recomendado\)](#)
 9. [Dicas e Troubleshooting](#)
-

1. O que é Docker?

Docker é uma plataforma para **criar, distribuir e executar aplicações dentro de contêineres** — ambientes isolados que empacotam o código junto com todas as suas dependências.

Por que usar Docker na disciplina de LBD?

Sem Docker, instalar um banco de dados como o MySQL pode ser uma tarefa problemática:

- Conflitos com versões já instaladas no sistema
- Dificuldades com configurações de sistema operacional
- "Funciona na minha máquina, mas não na sua"

Com Docker, você:

- Sobe um MySQL em **segundos**, com o comando certo
- Testa **diferentes versões** sem conflito
- Remove tudo sem deixar rastros no sistema
- Garante o **mesmo ambiente** para todos da turma

Contêineres vs. Máquinas Virtuais

Característica	Máquina Virtual (VM)	Contêiner Docker
Inicialização	Minutos	Segundos
Uso de RAM	Alto (GB)	Baixo (MB)
Sistema Operacional	SO completo por VM	Compartilha o kernel
Portabilidade	Limitada	Alta
Ideal para	Ambientes completos	Serviços isolados

2. Conceitos Fundamentais

Imagem (Image)

Uma **imagem** é como um "molde" ou "receita" somente leitura. Ela contém o sistema de arquivos, as dependências e as configurações necessárias para rodar uma aplicação. Imagens ficam armazenadas no **Docker Hub** (hub.docker.com) e são baixadas sob demanda.

Analogia: A imagem é como um ISO de sistema operacional.

Contêiner (Container)

Um **contêiner** é uma instância em execução de uma imagem. Você pode criar vários contêineres a partir da mesma imagem, e cada um é independente.

Analogia: Se a imagem é uma receita, o contêiner é o prato pronto.

Volume (Volume)

Um **volume** é o mecanismo para **persistir dados** fora do ciclo de vida do contêiner. Sem volume, quando você remove um contêiner MySQL, todos os seus dados são perdidos. Com volume, os dados sobrevivem.

Rede (Network)

A **rede do Docker** permite que contêineres se comuniquem entre si. Por padrão, cada contêiner é isolado. Para que o phpMyAdmin "fale" com o MySQL, ambos precisam estar na **mesma rede Docker**.

Dockerfile

Arquivo de texto com instruções para **construir uma imagem customizada**. Não precisaremos criar um Dockerfile nesta disciplina — usaremos imagens prontas do Docker Hub.

Docker Compose

Ferramenta para definir e gerenciar **múltiplos contêineres** via um arquivo YAML (`docker-compose.yml`). É a forma recomendada para subir MySQL + phpMyAdmin juntos.

Docker Hub

Repositório público de imagens Docker. É de lá que vêm as imagens do `mysql` , `phpmyadmin` , `postgres` , `mongo` , etc.

3. Instalando o Docker

Linux (Ubuntu/Debian)

```
# Atualizar pacotes
sudo apt update

# Instalar dependências
sudo apt install -y apt-transport-https ca-certificates curl software-properties-
```

```
common
```

```
# Adicionar a chave GPG oficial do Docker
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/docker-archive-keyring.gpg

# Adicionar repositório do Docker
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-
archive-keyring.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -cs)
stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Instalar Docker
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin

# Permitir usar Docker sem sudo (faça logout e login depois)
sudo usermod -aG docker $USER
```

Windows e macOS

Baixe o **Docker Desktop** em: <https://www.docker.com/products/docker-desktop>

Verificar instalação

```
docker --version
docker compose version
```

4. Comandos Essenciais do Docker

Gerenciamento de Contêineres

```
# Listar contêineres em execução
docker ps

# Listar TODOS os contêineres (incluindo parados)
docker ps -a

# Criar e iniciar um contêiner
docker run <imagem>

# Criar e iniciar em segundo plano (modo detached)
docker run -d <imagem>

# Criar com nome personalizado
docker run --name meu-container <imagem>

# Parar um contêiner
docker stop <id ou nome>
```

```
# Iniciar um contêiner parado
docker start <id ou nome>

# Reiniciar um contêiner
docker restart <id ou nome>

# Remover um contêiner (precisa estar parado)
docker rm <id ou nome>

# Forçar remoção mesmo que em execução
docker rm -f <id ou nome>

# Remover todos os contêineres parados de uma vez
docker container prune
```

Gerenciamento de Imagens

```
# Listar imagens locais
docker images

# Baixar uma imagem do Docker Hub
docker pull <imagem>:<tag>

# Exemplo: baixar MySQL 8.0
docker pull mysql:8.0

# Remover uma imagem local
docker rmi <imagem>

# Remover imagens não usadas
docker image prune
```

Inspeção e Logs

```
# Ver logs de um contêiner
docker logs <id ou nome>

# Acompanhar logs em tempo real
docker logs -f <id ou nome>

# Inspecionar detalhes de um contêiner (JSON completo)
docker inspect <id ou nome>

# Ver uso de recursos (CPU, RAM)
docker stats
```

Acessar o Terminal do Contêiner

```
# Abrir shell bash dentro de um contêiner em execução
docker exec -it <id ou nome> bash

# Ou usar sh caso bash não esteja disponível
docker exec -it <id ou nome> sh

# Sair do terminal do contêiner sem parar o contêiner
# Use: Ctrl + D ou exit
```

Gerenciamento de Redes

```
# Listar redes
docker network ls

# Criar uma rede
docker network create <nome-da-rede>

# Inspecionar uma rede
docker network inspect <nome-da-rede>

# Remover uma rede
docker network rm <nome-da-rede>
```

Gerenciamento de Volumes

```
# Listar volumes
docker volume ls

# Criar um volume
docker volume create <nome-do-volume>

# Inspecionar um volume
docker volume inspect <nome-do-volume>

# Remover volumes não utilizados
docker volume prune
```

Flags Importantes do `docker run`

Flag	Descrição	Exemplo
-d	Executa em segundo plano (detached)	<code>docker run -d mysql:8.0</code>
-p host:container	Mapeia porta do host para o contêiner	<code>-p 3306:3306</code>
-e VAR=valor	Define variável de ambiente	<code>-e MYSQL_ROOT_PASSWORD=senha</code>
-v host:container	Monta volume	<code>-v /dados:/var/lib/mysql</code>

--name	Define nome do contêiner	--name meu-mysql
--network	Conecta a uma rede específica	--network lbd-net
--rm	Remove o contêiner ao parar	docker run --rm ubuntu bash
-it	Modo interativo com terminal	docker run -it ubuntu bash

5. MySQL com Docker

Baixar a Imagem do MySQL

O Docker Hub oferece imagens oficiais do MySQL. Você pode escolher a versão desejada:

```
# MySQL 8.0 (versão estável mais recente)
docker pull mysql:8.0

# MySQL 5.7 (versão legada, ainda muito usada)
docker pull mysql:5.7

# MySQL mais recente (latest)
docker pull mysql:latest

# Listar versões baixadas
docker images mysql
```

Dica: Para ver todas as tags disponíveis, acesse: hub.docker.com/_/mysql

Iniciando o MySQL 8.0

```
docker run -d \
  --name mysql80 \
  -e MYSQL_ROOT_PASSWORD=root123 \
  -e MYSQL_DATABASE=lbd \
  -p 3306:3306 \
  mysql:8.0
```

Explicação de cada parâmetro:

- -d → roda em background
- --name mysql80 → nome do contêiner
- -e MYSQL_ROOT_PASSWORD=root123 → senha do usuário root
- -e MYSQL_DATABASE=lbd → cria automaticamente um banco chamado lbd
- -p 3306:3306 → expõe a porta 3306 para acesso externo
- mysql:8.0 → imagem e versão

Iniciando o MySQL 5.7

```
docker run -d \
  --name mysql57 \
  -e MYSQL_ROOT_PASSWORD=root123 \
  -e MYSQL_DATABASE=lbd \
  -p 3307:3306 \
  mysql:5.7
```

Note que usamos a porta `3307` no host para evitar conflito com o MySQL 8.0 rodando na `3306`.

Iniciando com MariaDB (alternativa ao MySQL)

```
docker run -d \
  --name mariadb \
  -e MYSQL_ROOT_PASSWORD=root123 \
  -e MYSQL_DATABASE=lbd \
  -p 3308:3306 \
  mariadb:10.11
```

MySQL com Dados Persistentes (Volume)

Por padrão, ao remover o contêiner, os dados do banco são perdidos. Para persistir os dados:

```
# Criar volume nomeado
docker volume create mysql-data

# Iniciar MySQL com volume
docker run -d \
  --name mysql80 \
  -e MYSQL_ROOT_PASSWORD=root123 \
  -e MYSQL_DATABASE=lbd \
  -p 3306:3306 \
  -v mysql-data:/var/lib/mysql \
  mysql:8.0
```

Agora, mesmo que você remova e recrie o contêiner, os dados serão preservados no volume `mysql-data`.

Acessando o MySQL via Terminal

```
# Acessar o cliente MySQL dentro do contêiner
docker exec -it mysql80 mysql -u root -p

# Após entrar, você verá o prompt:
# mysql>

# Comandos úteis dentro do MySQL:
# Listar bancos de dados:
SHOW DATABASES;
```

```
# Selecionar um banco:
USE lbd;

# Listar tabelas:
SHOW TABLES;

# Sair:
EXIT;
```

Verificando se o MySQL está Pronto

```
# Ver logs do MySQL (útil para debug)
docker logs mysql80

# Aguardar o MySQL iniciar (pode demorar alguns segundos)
docker logs -f mysql80
# Quando aparecer "ready for connections", está pronto!
```

6. phpMyAdmin com Docker

O **phpMyAdmin** é uma interface web para gerenciar bancos de dados MySQL visualmente.

Subindo o phpMyAdmin

```
docker run -d \
  --name phpmyadmin \
  -p 8080:80 \
  phpmyadmin:latest
```

Após isso, acesse: <http://localhost:8080>

Assim, o phpMyAdmin está rodando, mas ainda não está configurado para conectar no MySQL de outro contêiner. Veja a seção seguinte.

7. Conectando o phpMyAdmin ao MySQL

Para que o phpMyAdmin se comunique com o MySQL rodando em outro contêiner, os dois precisam estar na **mesma rede Docker**.

Passo 1 — Criar uma Rede Docker

```
docker network create lbd-net
```

Passo 2 — Iniciar o MySQL na Rede Criada

```
docker run -d \
  --name mysql80 \
  --network lbd-net \
  -e MYSQL_ROOT_PASSWORD=root123 \
  -e MYSQL_DATABASE=lbd \
  -p 3306:3306 \
  -v mysql-data:/var/lib/mysql \
  mysql:8.0
```

Passo 3 — Iniciar o phpMyAdmin na Mesma Rede

```
docker run -d \
  --name phpmyadmin \
  --network lbd-net \
  -e PMA_HOST=mysql80 \
  -e PMA_PORT=3306 \
  -p 8080:80 \
  phpmyadmin:latest
```

Explicação:

- `--network lbd-net` → conecta à mesma rede que o MySQL
- `-e PMA_HOST=mysql80` → informa o **nome do contêiner** MySQL (funciona como DNS interno)
- `-e PMA_PORT=3306` → porta do MySQL dentro da rede interna

Passo 4 — Acessar o phpMyAdmin

Abra o navegador e acesse: <http://localhost:8080>

- **Usuário:** root
- **Senha:** root123

Se tudo estiver correto, você verá a interface do phpMyAdmin conectada ao banco de dados MySQL!

Verificando a Conectividade

```
# Ver todos os contêineres rodando
docker ps

# Inspecionar a rede para ver os contêineres conectados
docker network inspect lbd-net

# Ver logs do phpMyAdmin em caso de erro
docker logs phpmyadmin
```

8. Usando Docker Compose (Recomendado)

O **Docker Compose** é a forma mais organizada e reproduzível de subir MySQL + phpMyAdmin juntos. Em vez de digitir vários comandos, você define tudo em um arquivo YAML.

Criando o `docker-compose.yml`

Crie uma pasta para o projeto e dentro dela crie o arquivo `docker-compose.yml` :

```
version: '3.8'

services:

# — Serviço MySQL —————
mysql:
  image: mysql:8.0          # Mude para mysql:5.7 se necessário
  container_name: lbd-mysql
  restart: unless-stopped
  environment:
    MYSQL_ROOT_PASSWORD: root123 # Senha do root
    MYSQL_DATABASE: lbd         # Banco criado automaticamente
    MYSQL_USER: aluno           # Usuário adicional (opcional)
    MYSQL_PASSWORD: aluno123    # Senha do usuário adicional
  ports:
    - "3306:3306"              # Porta acessível no host
  volumes:
    - mysql-data:/var/lib/mysql # Persistência dos dados
  networks:
    - lbd-net

# — Serviço phpMyAdmin —————
phpmyadmin:
  image: phpmyadmin:latest
  container_name: lbd-phpmyadmin
  restart: unless-stopped
  depends_on:
    - mysql          # Aguarda o MySQL iniciar primeiro
  environment:
    PMA_HOST: mysql    # Nome do serviço MySQL acima
    PMA_PORT: 3306
    PMA_ARBITRARY: 1    # Permite conectar em qualquer servidor
  ports:
    - "8080:80"        # phpMyAdmin acessível em localhost:8080
  networks:
    - lbd-net

# — Volumes —————
volumes:
  mysql-data:

# — Redes —————
networks:
```

```
lbd-net:
  driver: bridge
```

Comandos do Docker Compose

```
# Subir todos os serviços em segundo plano
docker compose up -d

# Ver o status dos serviços
docker compose ps

# Ver logs de todos os serviços
docker compose logs

# Ver logs de um serviço específico
docker compose logs mysql
docker compose logs phpmyadmin

# Acompanhar logs em tempo real
docker compose logs -f

# Parar os serviços (sem remover)
docker compose stop

# Iniciar serviços parados
docker compose start

# Parar e remover contêineres, redes (mantém volumes)
docker compose down

# Parar, remover contêineres, redes E volumes (APAGA OS DADOS!)
docker compose down -v
```

Atenção: `docker compose down -v` apaga o volume com os dados do banco. Use apenas quando quiser recomeçar do zero.

Usando MySQL 5.7 em vez de 8.0

Basta alterar a linha da imagem no `docker-compose.yml` :

```
# Troque:
image: mysql:8.0
# Por:
image: mysql:5.7
```

E execute:

```
docker compose down
```

```
docker compose up -d
```

9. Dicas e Troubleshooting

Problema: Porta 3306 já em uso

```
# Verificar o que está usando a porta
sudo lsof -i :3306

# Parar o MySQL local (se instalado diretamente no sistema)
sudo systemctl stop mysql

# Ou mudar a porta no docker run / docker-compose.yml para 3307
```

Problema: Contêiner MySQL reinicia em loop

```
# Ver o motivo do erro
docker logs lbd-mysql

# Causas comuns:
# - MYSQL_ROOT_PASSWORD não definida
# - Volume corrompido (tente: docker volume rm mysql-data)
```

Problema: phpMyAdmin não conecta no MySQL

```
# Verificar se ambos estão na mesma rede
docker network inspect lbd-net

# Verificar se o MySQL está pronto
docker logs lbd-mysql | grep "ready for connections"

# Testar conectividade entre contêineres
docker exec -it lbd-phpmyadmin ping lbd-mysql
```

Problema: Acesso negado (Access denied)

```
# Acessar o MySQL como root para verificar usuários
docker exec -it lbd-mysql mysql -u root -p

# Dentro do MySQL, verificar usuários:
SELECT user, host FROM mysql.user;

# Criar usuário se necessário:
CREATE USER 'aluno'@ '%' IDENTIFIED BY 'aluno123';
```

```
GRANT ALL PRIVILEGES ON lbd.* TO 'aluno'@'%';  
FLUSH PRIVILEGES;
```

Limpeza Completa do Ambiente

```
# Parar e remover todos os contêineres do compose  
docker compose down -v  
  
# Ou manualmente:  
docker stop mysql80 phpmyadmin  
docker rm mysql80 phpmyadmin  
docker volume rm mysql-data  
docker network rm lbd-net
```

Resumo Rápido — Fluxo Completo

```
# 1. Criar rede  
docker network create lbd-net  
  
# 2. Subir MySQL 8.0  
docker run -d \  
  --name lbd-mysql \  
  --network lbd-net \  
  -e MYSQL_ROOT_PASSWORD=root123 \  
  -e MYSQL_DATABASE=lbd \  
  -p 3306:3306 \  
  -v mysql-data:/var/lib/mysql \  
  mysql:8.0  
  
# 3. Subir phpMyAdmin  
docker run -d \  
  --name lbd-phpmyadmin \  
  --network lbd-net \  
  -e PMA_HOST=lbd-mysql \  
  -p 8080:80 \  
  phpmyadmin:latest  
  
# 4. Acessar: http://localhost:8080  
#   Usuário: root | Senha: root123
```

Referências

- [Documentação oficial do Docker](#)
 - [Docker Hub — MySQL](#)
 - [Docker Hub — phpMyAdmin](#)
 - [Docker Compose — Reference](#)
 - Livro: *Introdução a Docker* (material disponível no Moodle da disciplina)
-

Material preparado para a disciplina de Laboratório de Banco de Dados (LBD) Engenharia de Software — Universidade Federal de Mato Grosso do Sul (UFMS)